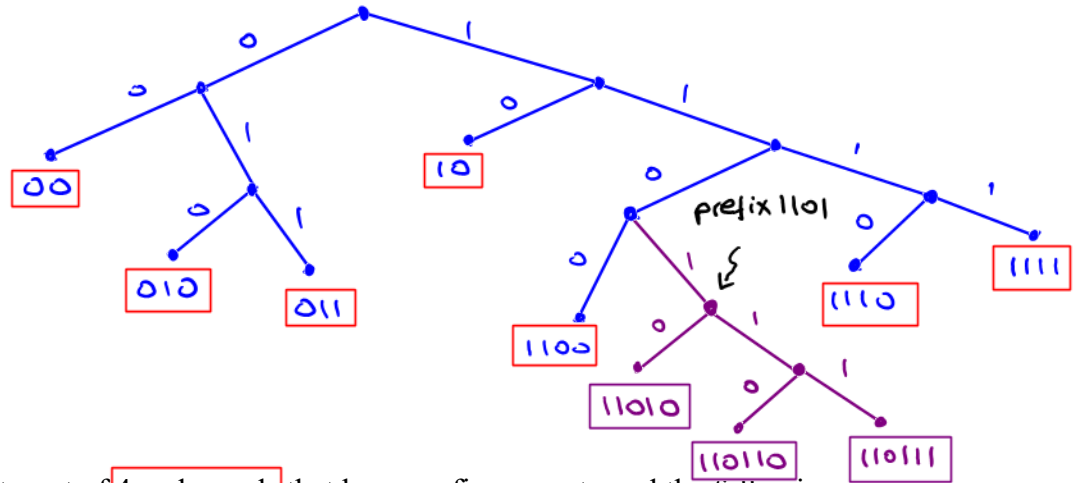


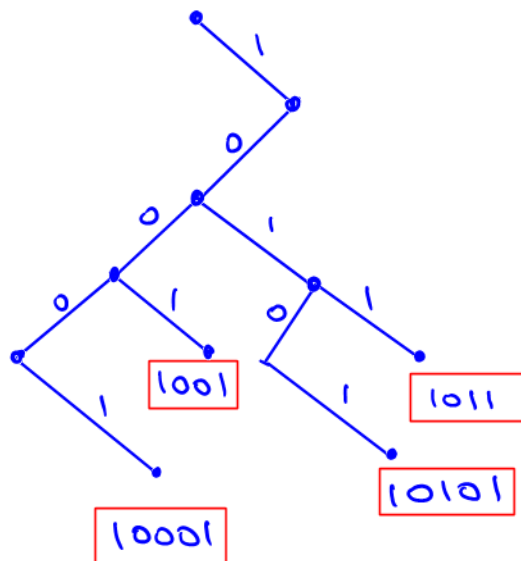
1. Verify that $S = \{00, 10, 1100, 1111, 010, 011, 1110\}$ is the set of the codewords that have prefix property. Expand the set with three additional codewords.

all codewords
are leaves



2. Construct a set of 4 codewords that have prefix property and the following additional constraints:

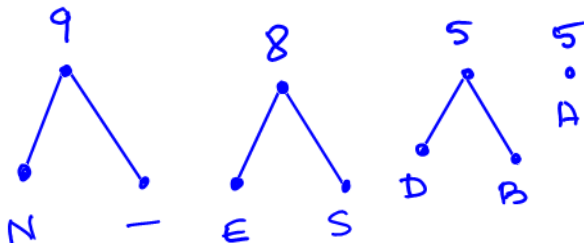
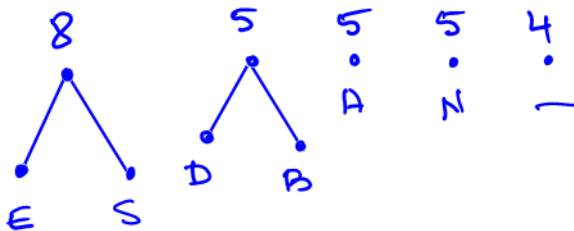
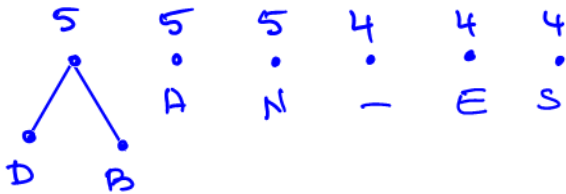
- No codeword can start with 0 or 11,
- No codeword can end with 0,
- Exactly two codewords have a length of 4.



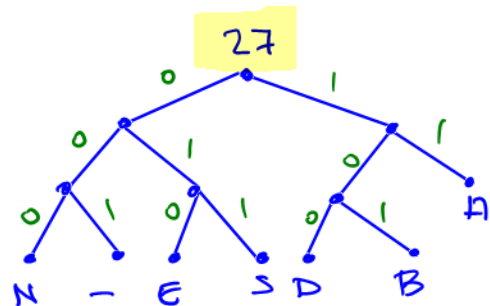
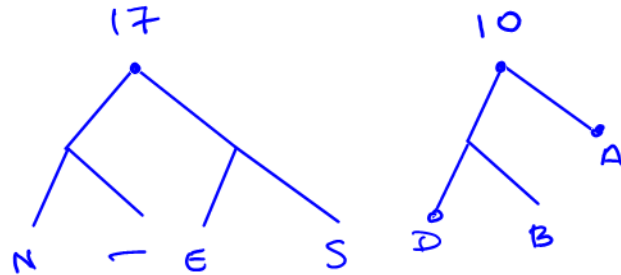
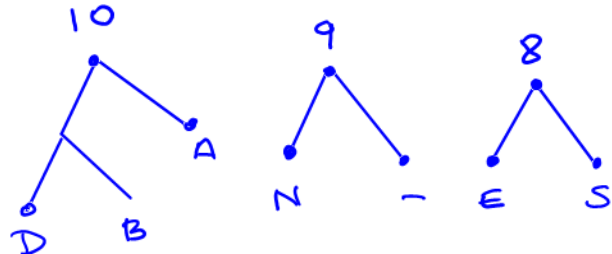
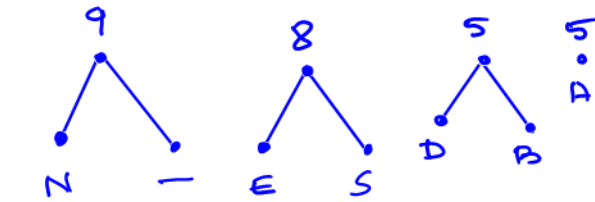
3. Build Huffman tree based on "send beans and banana seeds" and provide codeword for each character. What is the total number of bits required to encode the sentence.

4 SSSS
 4 EEEE
 5 NNNNN
 3 DDD
 4 - - - -
 2 BB
 5 AAAAA
27

5 5 4 4 4 3 2
 A N - E S D B



*
*



char	Cw	freq	# of bits
N	000	5	5 x 3 = 15
S	011	4	4 x 3 = 12
E	010	4	4 x 3 = 12
-	001	4	4 x 3 = 12
D	100	3	3 x 3 = 9
B	101	2	2 x 3 = 6
A	11	5	5 x 2 = 10

+ = 76 bits in total

4. A certain file contains only two characters, A and B:

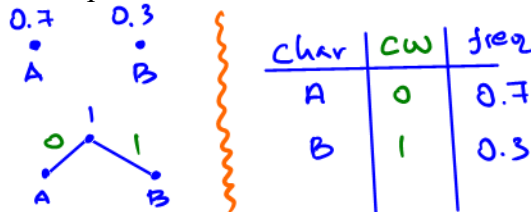
Char	Normalized Frequency
A	0.7
B	0.3

According to Shannon's Formula the lower bound on the number of bits per character is:

$$\sum_i^n p_i \log_2 \left(\frac{1}{p_i} \right) = 0.7 \cdot \log_2 \left(\frac{1}{0.7} \right) + 0.3 \cdot \log_2 \left(\frac{1}{0.3} \right) = 0.88129$$

can be done better

- a) Build Huffman tree and calculate the expected number of bits per character. Compare this result with the lower bound.

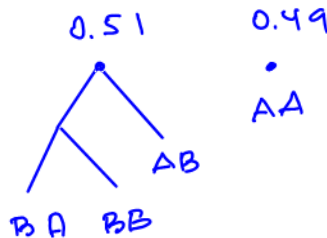


expected # of bits per 1 char : $(0.7)(1) + (0.3)(1) = 1$

- b) Based on the original information calculate the normalized frequency of symbols composed of two characters.

Build Huffman tree and calculate the expected number of bits per symbol and expected number of bits per character. Compare this result with the lower bound.

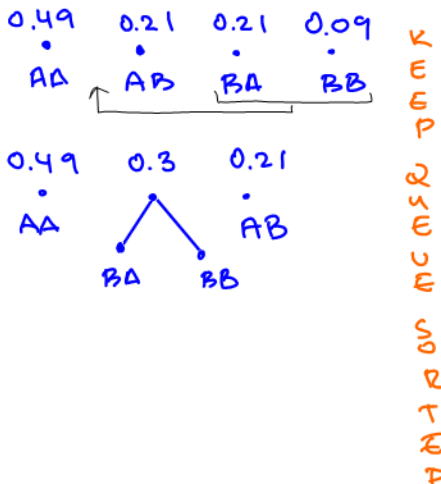
2 char	freq
AA	$(0.7)(0.7) = 0.49$
AB	$(0.7)(0.3) = 0.21$
BA	$(0.3)(0.7) = 0.21$
BB	$(0.3)(0.3) = 0.09$



2 char	freq	CW	# of bits
BA	0.21	000	$(0.21)(3)$
BB	0.09	001	$(0.09)(3)$
AB	0.21	01	$(0.21)(2)$
AA	0.49	1	$(0.49)(1)$

$1.81 \frac{\text{bits}}{2 \text{ char}}$

then $\frac{1.81}{2} = 0.905 \frac{\text{bits}}{1 \text{ char}}$



Example of Recursive Algorithm and Mathematical Induction.

Example of 4x4 board with one square missing, covered with L-shapes

1	1	2	2
1	3		2
4	3	3	5
4	4	5	5

For any integer $n \geq 1$, if any one square is removed from a $2^n \times 2^n$ checkerboard, the remaining squares can be covered with L-shaped pieces that cover three squares (i.e. trominoes).

- 1) BASE CASE: verify that the statement is true for $n=1$
If any square is removed from $2^1 \times 2^1$ checkerboard the remaining squares can be covered with L-shaped pieces (trominoes)



verified ✓

- 2) Assume that the following is true for $n=k$
if any square is removed from $2^k \times 2^k$ checkerboard the remaining squares can be covered with trominoes

- 3) Target for $n=k+1$.
if any square is removed from $2^{k+1} \times 2^{k+1}$ checkerboard the remaining squares can be covered with trominoes

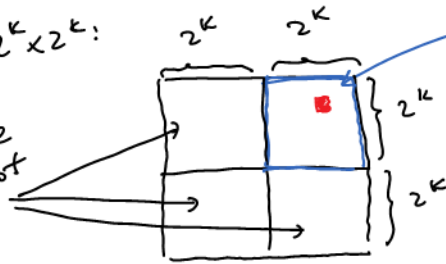
④ proof:

Consider the board



this checkerboard can be divided in 4 checker boards $2^k \times 2^k$:

they don't have
tromino, so can not
use step 2



this board
can be covered
by I.H. (i.e. step 2)

place one tromino such that

now I.H. applies
to every board
is proved

